

Flying into the Quantum Era: Secure UAV Communication with Expert Policy Scheduling

[Author Names Redacted for Review]

[Affiliation Redacted for Review]

Abstract—Unmanned Aerial Vehicles (UAVs) now operate in safety-critical roles, but MAVLink command-and-control traffic remains vulnerable to eavesdropping, injection, and replay when left unprotected. This risk is amplified by the quantum transition: Shor’s algorithm threatens currently deployed public-key primitives, making “store-now, decrypt-later” collection a practical concern. We present SECURETUNNEL, a deployment-ready post-quantum cryptographic tunnel for MAVLink links between a drone companion computer and a ground control station. The design provides a 72-suite registry from a level-consistent product of nine KEM parameter sets (ML-KEM, HQC, Classic McEliece), eight signature parameter sets (ML-DSA, Falcon, SPHINCS⁺), and three AEAD ciphers (AES-256-GCM, ChaCha20-Poly1305, Ascon-128a). Its two-message handshake combines KEM-based key establishment, signature-authenticated server identity, and HMAC-based pre-shared-key drone verification. We evaluate all 72 suites on a Raspberry Pi 4 with a Pixhawk flight controller across baseline, XGBoost-detection, and Transformer-detection phases, reporting handshake latency and MAVLink integrity from the canonical full run. In that run, ML-KEM-768 with ML-DSA-65 completes in 17.811 ms, while the fastest verified suite (ML-KEM-512 with Falcon-512, AES-GCM) reaches 9.391 ms. To maintain operation under resource stress, MDEAS jointly controls AEAD choice, NIST level, and detector intensity using runtime telemetry with conservative detector-overhead priors. Overall, the paper contributes a complete UAV PQC suite space, an auditable 72-suite measurement model, and a practical resource-aware scheduling architecture for embedded deployment.

Index Terms—Post-Quantum Cryptography, UAV Security, MAVLink, Key Encapsulation Mechanism, Digital Signature, Authenticated Encryption, Embedded Systems, DDoS Detection, Crypto Agility

I. INTRODUCTION

Unmanned Aerial Vehicles have moved from niche platforms to mainstream infrastructure across precision agriculture, inspection, emergency response, and logistics [27]. As deployment density grows in both regulated and unregulated airspace, communication security becomes an operational requirement rather than an optional add-on. At the core of most small UAV systems lies *MAVLink* [29], a lightweight binary telemetry protocol that carries flight commands, sensor data, and heartbeat signals between a flight controller and a ground control station (GCS). MAVLink v2 introduced message signing via a keyed SHA-256 construction (truncated to 48 bits), yet it lacks key establishment, forward secrecy, and confidentiality—any adversary with network proximity can observe, inject, or replay commands [28].

This vulnerability is compounded by the looming quantum threat. Shor’s algorithm [16] breaks RSA, DSA, and elliptic-curve Diffie–Hellman in polynomial time on a sufficiently large quantum computer; Grover’s algorithm [17] halves the effective security of symmetric ciphers. The National Institute of Standards and Technology (NIST) has responded by standardising post-quantum algorithms: ML-KEM (FIPS 203) [1] for key encapsulation, ML-DSA (FIPS 204) [2] for digital signatures, and SLH-DSA (FIPS 205) [3] as a hash-based signature alternative. NIST’s fourth-round status report [4] discusses additional code-based candidates; in particular, HQC remains on the standardization path, while Classic McEliece is retained here as a conservative benchmark family [9], [10]. Despite this standardisation progress, most UAV-focused studies still evaluate isolated primitives or partial protocol steps, rather than a deployment-ready system that combines suite agility, measured runtime cost, and adaptive scheduling.

A. Problem Statement and Research Gap

Existing embedded PQC benchmarks [22], [23] report primitive timings in isolation, but not the compound cost of complete handshakes over live MAVLink traffic. PQC tunnel prototypes such as Rosenpass [25] target VPN scenarios and do not address UAV-specific constraints: strict latency budgets, thermal limits, battery variability, and in-flight crypto adaptation. Meanwhile, drone security studies [27], [28] clearly identify MAVLink risk, but stop short of a complete, measured, and adaptive end-to-end deployment.

B. Contributions

This paper makes the following contributions:

- C1 PQC Suite Registry.** A 72-suite registry from the NIST-level-consistent product of 9 KEMs $\times$ 8 signatures $\times$ 3 AEADs, with a two-message handshake that provides forward secrecy, mutual authentication, and anti-downgrade binding.
- C2 Measurement-Driven Cost Model.** A comprehensive benchmark of all 72 suites on a Raspberry Pi 4 companion computer with Pixhawk flight controller, measuring handshake latency, per-packet AEAD cost, and MAVLink integrity across baseline and two DDoS-detection configurations, with explicit reporting boundaries when sensor-complete power telemetry is unavailable.

C3 Three-Axis Expert Scheduler (MDEAS). A measurement-driven scheduler that jointly optimises AEAD selection (data-plane axis), NIST security level (control-plane axis), and DDoS detection intensity (compute-plane axis) using telemetry-aware gates, thermal prediction, and break-even logic.

C4 Deployable System. A complete, end-to-end implementation operating as a transparent “bump-in-the-wire” proxy requiring no firmware modification, validated with live ArduPilot telemetry over WiFi.

To keep quantitative claims auditable, all benchmark values reported herein are traceable to the archived Pi 4 run artifacts (run tag 20260220_full).

C. Paper Organisation

Section II surveys related work. Section III defines the threat model. Section IV describes the post-quantum system architecture. Section V details the experimental setup. Section VI presents results and analysis. Section VII discusses trade-offs and limitations. Section VIII concludes with future directions.

II. RELATED WORK

A. Post-Quantum Cryptography for Embedded Systems

The pqm4 project [22] provides optimised ARM Cortex-M4 implementations of NIST PQC candidates, establishing micro-benchmark baselines for key generation, encapsulation, and signing on bare-metal platforms. Kannwischer et al. [23] extended this to Cortex-A processors, reporting ML-KEM-768 key generation in 0.33 ms on a Cortex-A72 at 1.8 GHz. These works benchmark primitives in isolation; they do not measure compound handshake cost over live protocol sessions, nor do they capture full runtime behavior under an adaptive scheduler. Our work evaluates all 72 suites end-to-end on a Raspberry Pi 4 (Cortex-A72) with live MAVLink traffic, separating canonical suite outcomes from supporting primitive profiles to keep claims auditable.

B. PQC in Network Protocols

Post-quantum migration has been studied in TLS 1.3, where Paquin et al. [24] evaluated hybrid key exchange using liboqs [19]. Rosenpass [25] adds a PQC handshake layer atop WireGuard, using Classic McEliece for long-term keys and Kyber for ephemeral exchange. Hülising et al. [26] analysed PQC overhead in IPsec and OpenVPN deployments. These protocol-level studies target server-class hardware with ample resources. In contrast, our tunnel targets a 4 W embedded companion computer with thermal constraints and battery awareness, requiring an adaptive scheduling layer absent in prior protocol work.

The Open Quantum Safe (OQS) project [19], [20] provides the liboqs C library and its Python wrapper, implementing all NIST PQC finalists and additional candidates. We build directly on liboqs-python for all KEM and signature operations, using version 0.14.0 with ARM NEON optimisations enabled.

C. UAV Security and MAVLink Protection

Altawy and Youssef [27] surveyed security, privacy, and safety aspects of civilian drones, identifying MAVLink’s lack of encryption as a critical vulnerability. Pham and Vakili [28] analysed post-quantum cryptography for UAV communication, evaluating KEM and signature overhead on ARM processors but without a complete tunnel implementation or runtime adaptation. The MAVLink specification [29] defines optional message signing (a keyed SHA-256 construction, not HMAC) in version 2.0, but this provides only integrity, not confidentiality or forward secrecy. Our system provides a *complete* transparent tunnel: KEM-based key establishment, signature-verified authentication, AEAD-encrypted data plane, and runtime suite adaptation—all without modifying the flight controller firmware.

D. DDoS Detection on Edge Devices

Lightweight DDoS detection on embedded platforms has been explored using ensemble methods (XGBoost, Random Forest) and deep learning architectures. Sharafaldin et al. [34] introduced the CIC-IDS dataset widely used for intrusion detection benchmarking. More recent work has applied Transformer-based models to network traffic classification [35], achieving higher accuracy at a significant computational cost. Our system integrates two detector tiers: an XGBoost model with 54 features and ≈ 7 ms inference time, and a Time Series Transformer (TST) model with 46 features and ≈ 11 ms inference time. In this manuscript revision, detector thermal and power effects in the canonical run are reported as *unknown* when telemetry is absent; the scheduler therefore uses conservative configured overhead priors (0.95 W/4.8 °C for XGBoost, 1.97 W/10.7 °C for TST) for safe runtime control.

E. Crypto Agility and Runtime Adaptation

Crypto agility—the ability to switch algorithms at runtime—is recognised as essential for post-quantum migration [4]. However, most systems treat it as a configuration-time choice rather than a runtime decision. Barker [36] recommends algorithm transitions but does not address the dynamic context of resource-constrained flight. Our MDEAS scheduler treats crypto agility as a *continuous optimisation problem* with three axes (AEAD, security level, detector intensity), using hysteresis, break-even analysis, and thermal prediction to avoid oscillation while responding to changing conditions.

III. THREAT MODEL

We consider four adversary classes with increasing capability, following the Dolev–Yao model [18] extended with quantum computation.

A. Adversary Classes

A₁: Passive Eavesdropper. Captures all network traffic on the wireless link between drone and GCS. Can perform “store now, decrypt later” for future quantum decryption.

$\mathcal{A}_2$: Active Network Adversary. Controls the network channel (Dolev–Yao model): can intercept, modify, replay, delay, and inject packets. Can mount man-in-the-middle attacks, injection of rogue MAVLink commands, and GPS spoofing via crafted `GLOBAL_POSITION_INT` messages.

$\mathcal{A}_3$: DDoS Adversary. Floods the encrypted channel with malformed or replayed packets to exhaust computational resources on the companion computer, degrading real-time telemetry delivery.

$\mathcal{A}_4$: Quantum Adversary. Possesses a Cryptographically Relevant Quantum Computer (CRQC) capable of running Shor’s algorithm against RSA, DSA, and ECDH, and Grover’s algorithm to accelerate brute-force search against symmetric ciphers.

B. Trust Assumptions

- T1** The flight controller firmware (ArduPilot [31]) and companion OS are trusted.
- T2** A pre-shared key (PSK) is securely provisioned on the drone before deployment.
- T3** The GCS signing key pair is generated offline and the public key is distributed to the drone.
- T4** The `liboqs` [19] library correctly implements the NIST PQC algorithms.
- T5** Physical access to the companion computer during flight is infeasible.

C. Security Goals

- G1 Confidentiality.** All MAVLink traffic between drone and GCS is encrypted with AEAD, rendering eavesdropping (including quantum-assisted “store now, decrypt later”) ineffective.
- G2 Integrity and Authenticity.** Every packet carries an AEAD authentication tag bound to the 22-byte header as associated data; any modification is detected.
- G3 Replay Protection.** A sliding-window replay filter (default window $W = 1024$, following RFC 6479 / WireGuard pattern) rejects duplicate sequence numbers.
- G4 Forward Secrecy.** Ephemeral KEM key pairs are generated per session; compromise of long-term keys does not reveal past session keys.
- G5 Mutual Authentication.** The GCS proves its identity via digital signature; the drone proves its identity via HMAC over PSK.
- G6 Anti-Downgrade.** The wire version byte is included in the signed transcript, preventing protocol version rollback.
- G7 Availability under DDoS.** Integrated ML-based detection identifies volumetric attacks; the scheduler downgrades to lighter suites to maintain telemetry throughput.

D. Out of Scope

The following are explicitly outside this paper’s guarantee boundary:

- O1** Physical compromise of the drone, companion computer, or GCS host.

TABLE I: Threat–Mitigation Mapping

Adversary	Countermeasure
$\mathcal{A}_1$ (Eavesdrop)	AEAD encryption (G1), PQ KEM key exchange
$\mathcal{A}_2$ (Active)	Digital signature auth (G5), AEAD integrity (G2), replay window (G3), anti-downgrade (G6)
$\mathcal{A}_3$ (DDoS)	ML detectors, scheduler degradation (G7), rate-limited handshake
$\mathcal{A}_4$ (Quantum)	NIST PQC KEMs and signatures (G1, G4, G5)

- O2** Compromise of trusted software dependencies (e.g., `liboqs`, OS kernel, Python runtime).
- O3** RF-layer jamming and GNSS spoofing resilience beyond the encrypted MAVLink transport.
- O4** Long-horizon fleet key management and certificate life-cycle policy for multi-drone swarms.

Table I maps each adversary to the system mechanisms that counter it.

IV. POST-QUANTUM SYSTEM ARCHITECTURE

A. System Overview

SECURETUNNEL operates as a transparent “bump-in-the-wire” proxy that requires no modification to the flight controller firmware or ground station software. Fig. 1 illustrates the system topology.

The drone-side companion computer (Raspberry Pi 4) runs MAVProxy [30] connected to the Pixhawk flight controller [33] via USB serial. MAVProxy forwards telemetry to the PQC proxy over local UDP (ports 47003/47004). The proxy encrypts each packet with the negotiated AEAD cipher using the session key derived during the PQC handshake, transmits it over WiFi on the encrypted UDP channel (ports 46011/46012), and the GCS-side proxy decrypts and delivers to the ground MAVProxy instance. A TCP control channel (port 48080) carries scheduler commands, clock synchronisation, and rekey negotiation between the drone-driven scheduler and the GCS follower.

B. Suite Registry Design

The suite registry is the combinatorial backbone of SECURETUNNEL. Each suite is a triple (KEM, SIG, AEAD) where the KEM and SIG must share the same NIST security level. We include algorithms from three KEM families and three signature families at three NIST levels, crossed with three AEAD ciphers.

NIST Level Mapping.

- **L1** ($\approx$ AES-128 security): ML-KEM-512, HQC-128, Classic-McEliece-348864, ML-DSA-44¹, Falcon-512, SPHINCS⁺-128s.

¹ML-DSA-44 is classified as NIST Level 2 in FIPS 204; we map it to L1 for practical pairing with L1 KEMs following the rationale that L2 exceeds L1 by a small margin.

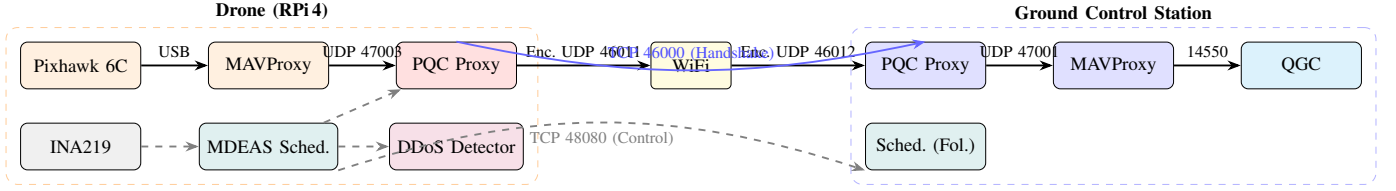


Fig. 1: End-to-end system architecture of SECURETUNNEL. Solid arrows show the data path; dashed arrows show control/monitoring relationships. The encrypted UDP channel carries AEAD-protected MAVLink packets; the TCP control channel carries scheduler commands, clock sync, and rekey negotiation.

TABLE II: KEM Algorithm Parameters (from liboqs 0.14.0)

Algorithm	Level	PK (B)	CT (B)	SS (B)
ML-KEM-512	L1	800	768	32
ML-KEM-768	L3	1 184	1 088	32
ML-KEM-1024	L5	1 568	1 568	32
HQC-128	L1	2 249	4 433	64
HQC-192	L3	4 522	9 026	64
HQC-256	L5	7 245	14 469	64
McEliece-348864	L1	261 120	96	32
McEliece-460896	L3	524 160	156	32
McEliece-8192128	L5	1 357 824	208	32

TABLE III: Signature Algorithm Parameters (from liboqs 0.14.0)

Algorithm	Level	PK (B)	SK (B)	Sig (B)
ML-DSA-44	L1	1 312	2 560	2 420
ML-DSA-65	L3	1 952	4 032	3 309
ML-DSA-87	L5	2 592	4 896	4 627
Falcon-512	L1	897	1 281	≤ 666
Falcon-1024	L5	1 793	2 305	$\leq 1 280$
SPHINCS ⁺ -128s	L1	32	64	7 856
SPHINCS ⁺ -192s	L3	48	96	16 224
SPHINCS ⁺ -256s	L5	64	128	29 792

- **L3** ($\approx$ AES-192 security): ML-KEM-768, HQC-192, Classic-McEliece-460896, ML-DSA-65, SPHINCS⁺-192s.
- **L5** ($\approx$ AES-256 security): ML-KEM-1024, HQC-256, Classic-McEliece-8192128, ML-DSA-87, Falcon-1024, SPHINCS⁺-256s.

The level-consistent pairing produces:

$$|S| = \underbrace{(3 \times 3)}_{L1} + \underbrace{(3 \times 2)}_{L3} + \underbrace{(3 \times 3)}_{L5} = 24 \quad (1)$$

KEM $\times$ SIG pairs (L3 has no Falcon variant), each crossed with 3 AEADs:

$$N_{\text{suites}} = 24 \times 3 = 72 \quad (2)$$

Each suite is assigned a canonical identifier of the form `cs-{kem}-{aead}-{sig}` (e.g., `cs-mlkem768-aesgcm-mldsa65`).

Table II summarises the KEM parameters, and Table III summarises the signature parameters.

C. KEM-Based Key Establishment

The handshake uses a two-message protocol over TCP (port 46000):

Message 1: ServerHello (GCS $\rightarrow$ Drone). The GCS generates an ephemeral KEM key pair $(pk_{\text{kem}}, sk_{\text{kem}}) \leftarrow \text{KEM.KeyGen}()$, a random 8-byte session identifier sid , and a random 8-byte challenge c . It constructs the transcript:

$$T = v \parallel \text{pq-drone-gcs:v1} \parallel sid \parallel \text{kem_name} \parallel \text{sig_name} \parallel pk_{\text{kem}} \parallel c \quad (3)$$

where v is the frozen wire version byte. The GCS signs the transcript: $\sigma_{\text{gcs}} \leftarrow \text{SIG.Sign}(sk_{\text{gcs}}, T)$ and sends the length-prefixed wire message containing all fields plus σ_{gcs} .

Message 2: ClientReply (Drone $\rightarrow$ GCS). The drone verifies σ_{gcs} using the pre-distributed GCS public key, checks that the negotiated algorithms match the expected suite (anti-downgrade), and computes:

$$(ct, K) \leftarrow \text{KEM.Encaps}(pk_{\text{kem}}) \quad (4)$$

where K is the shared secret. It also computes an HMAC authentication tag over the entire ServerHello wire:

$$\tau = \text{HMAC-SHA256}(PSK, \text{ServerHello}_{\text{wire}}) \quad (5)$$

and sends (ct, τ) .

Key Derivation. Both sides derive directional transport keys using HKDF-SHA256 [12]:

$$(k_{d \rightarrow g} \parallel k_{g \rightarrow d}) = \text{HKDF-SHA256}(K, s, \text{info}, 64) \quad (6)$$

where the salt $s = \text{"pq-drone-gcs|hkd f|v1"}$ and $\text{info} = \text{"pq-drone-gcs:kdf:v1|" } \parallel sid \parallel \text{kem_name} \parallel \text{sig_name}$. The shared secret K is zeroed immediately after derivation, following the WireGuard pattern.

Fig. 2 illustrates the complete two-message handshake.

D. AEAD Data-Plane Framing

Each plaintext MAVLink packet is encrypted under the negotiated AEAD cipher. The wire format is:

$$\text{frame} = \underbrace{H}_{22 \text{ B header}} \parallel \text{AEAD.Enc}(k, iv, pt, H) \quad (7)$$

The 22-byte header H is structured as:

$$H = (v, kem_id, kem_p, sig_id, sig_p, sid^8, seq^8, epoch)$$

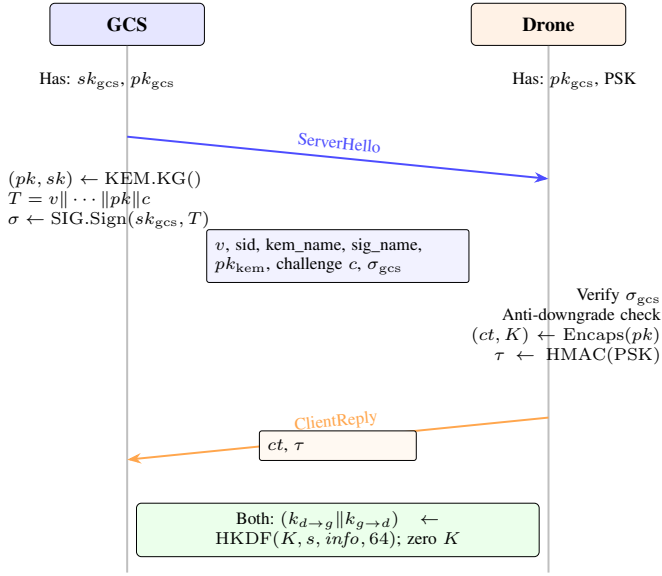


Fig. 2: Two-message post-quantum handshake protocol. The GCS authenticates via digital signature; the drone authenticates via HMAC over PSK. Both sides derive directional transport keys from the KEM shared secret.

and is bound as associated authenticated data (AAD) to the AEAD operation. The 12-byte initialisation vector is deterministically constructed from the epoch byte and sequence number, avoiding IV transmission and saving 12 bytes per packet:

$$iv = epoch || seq_{11B} \quad (8)$$

Replay protection follows the WireGuard/RFC 6479 sliding-window approach with a configurable window W (default 1024). The implementation uses a split *check/commit* pattern: the replay window is checked *before* AEAD decryption and committed *after* successful authentication, preventing a malicious packet from shifting the window without valid authentication.

Fig. 3 illustrates the complete AEAD wire format.

E. Rekey Protocol

Two rekey modes are supported:

Cold Restart. The proxy process is terminated, a new handshake is performed with the target suite, and a fresh proxy

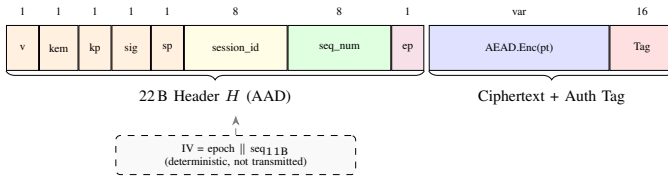


Fig. 3: AEAD wire format. The 22-byte header serves as AAD bound to the AEAD encryption. The IV is deterministically derived from epoch and sequence number, eliminating IV transmission overhead.

is started. This mode provides the strongest isolation between suites, as no key material persists across the restart boundary. It incurs a brief blackout ($\sim 2s$) during transition.

In-Band Rekey. The proxy remains running; the scheduler sends a `prepare_rekey` control message (packet type 0x02) through the encrypted channel. Both sides execute a new handshake on the TCP port (serialised with a lock to avoid port conflicts), atomically swap the AEAD Sender/Receiver objects, and zero the old key material. This minimises blackout but requires careful coordination via a two-phase commit protocol:

- 1) Coordinator $\rightarrow$ Follower: `prepare_rekey(suite, rid)`
- 2) Follower $\rightarrow$ Coordinator: `prepare_ok(rid)` or `prepare_fail`
- 3) Coordinator $\rightarrow$ Follower: `commit_rekey(suite, rid)`
- 4) Both execute handshake $\rightarrow$ atomic cipher swap

Fig. 4 illustrates both rekey modes.

The rekey overhead fraction for a session of duration R with handshake time T_{hs} is:

$$\Phi(R) = \frac{T_{hs}}{R + T_{hs}} \quad (9)$$

For the default suite (ML-KEM-768 + ML-DSA-65, $T_{hs} = 17.811 \text{ ms}$) with a 60-second rekey interval, $\Phi = 0.030\%$.

F. DDoS Detection Pipeline

The DDoS detection pipeline runs on the companion computer alongside the PQC proxy. Two detection tiers are available:

XGBoost Detector. A gradient-boosted tree model operating on 54 extracted features per traffic window. Inference takes $\sim 7 \text{ ms}$ per window. In this manuscript version, XGBoost overhead is treated as a configured policy prior rather than a measured cross-suite thermal/power delta, because canonical power telemetry is unavailable.

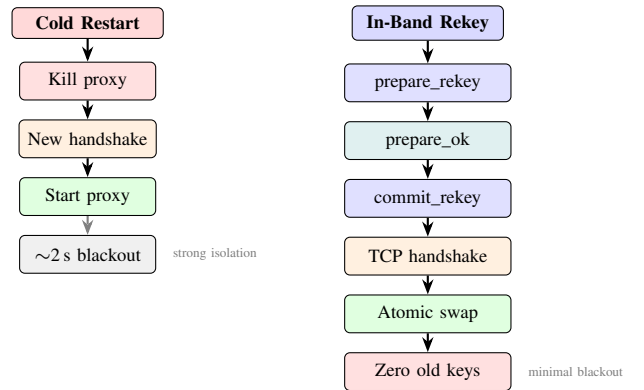


Fig. 4: Rekey protocol modes. Cold restart (left) provides full key isolation with $\sim 2s$ blackout. In-band rekey (right) uses a two-phase commit to atomically swap cipher state with minimal disruption.

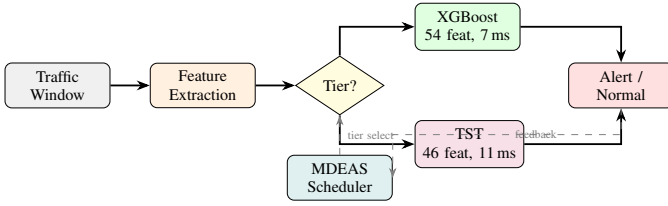


Fig. 5: Two-tier DDoS detection pipeline. The MDEAS scheduler selects the active detector tier based on thermal/compute budget. Only one detector operates at a time; detection alerts feed back into scheduler decisions.

Time Series Transformer (TST) Detector. A Transformer-based intrusion detection model operating on 46 features. Inference takes ~ 11 ms per window. Likewise, TST overhead is handled through configured safety margins in the scheduler model.

The scheduler maintains conservative safety margins for detector overhead (0.95 W/ 4.8°C for XGBoost, 1.97 W/ 10.7°C for TST) in its `DetectorOverhead` configuration, ensuring headroom for worst-case conditions.

Fig. 5 illustrates the two-tier detection pipeline architecture.

Only one detector runs at any time (the companion computer lacks resources for concurrent operation). The scheduler manages transitions between detector levels using the same process-group lifecycle management as proxy processes.

G. Three-Axis Expert Scheduler (MDEAS)

The Measurement-Driven Energy-Aware Scheduler (MDEAS) treats crypto agility as a joint optimisation problem over three axes:

Axis 1 — AEAD Selection (data-plane, per-packet cost). Selects among {ChaCha20-Poly1305, AES-256-GCM, Ascon-128a} based on measured per-packet encrypt/decrypt latency, temperature, and CPU utilisation. The AEAD cost profile is maintained as an exponentially weighted moving average (EWMA) with $\alpha = 0.2$ (steady state) and pre-seeded from benchmark data (Benchmark-Seeded Initialisation):

$$\hat{C}_{n+1} = \alpha \cdot C_{\text{obs}} + (1 - \alpha) \cdot \hat{C}_n \quad (10)$$

Axis 2 — NIST Security Level (control-plane, per-handshake cost). Selects among {L1, L3, L5} with a fixed KEM+SIG pairing per level: L1 $\rightarrow$ (ML-KEM-512, ML-DSA-44), L3 $\rightarrow$ (ML-KEM-768, ML-DSA-65), L5 $\rightarrow$ (ML-KEM-1024, ML-DSA-87).

Axis 3 — DDoS Detection Level (compute-plane, background cost). Selects among {None, XGBoost, TST} based on threat indicators with awareness of detector overhead.

Decision Cascade. The scheduler evaluates a 12-gate priority cascade at each tick:

1. Telemetry stale (> 2 s) $\rightarrow$ HOLD
2. DDoS detected (drop ratio > 0.10) $\rightarrow$ EMERGENCY
3. Battery critical (< 14.0 V) $\rightarrow$ EMERGENCY
4. Temperature critical ($> 80^\circ\text{C}$) $\rightarrow$ EMERGENCY
5. Emergency active $\rightarrow$ kill detector

6. Detector thermal ceiling exceeded $\rightarrow$ DOWN-GRADE_DETECTOR
7. Predictive thermal crossing (60 s extrapolation) $\rightarrow$ SWITCH_AEAD
8. Thermal/CPU stress $\rightarrow$ SWITCH_AEAD (with break-even)
9. AEAD recovery (stress subsided) $\rightarrow$ SWITCH_AEAD
10. Link degraded ($\text{gap}_{p95} > 1$ s) $\rightarrow$ DOWN-GRADE_LEVEL
11. Stable conditions, proactive rekey due $\rightarrow$ REKEY
12. Stable, upgrade hysteresis satisfied $\rightarrow$ UPGRADE

Break-Even Analysis. Before triggering an AEAD switch, the scheduler computes the break-even time:

$$t_{\text{be}} = \frac{C_{\text{rekey}}}{(\hat{C}_{\text{cur}} - \hat{C}_{\text{tgt}}) \cdot r_{\text{pkt}} \cdot 2} \quad (11)$$

where $C_{\text{rekey}} = 300$ ms is the handshake cost, $\hat{C}$ are the EWMA AEAD costs, and r_{pkt} is the packet rate. The switch is executed only if $t_{\text{be}} < 30$ s (nominal) or $t_{\text{be}} < 15$ s (under stress).

Hysteresis. Downgrade requires 5 s of sustained adverse conditions; upgrade requires 30 s of nominal conditions (tripled to 90 s when the vehicle is armed). This prevents oscillation during transient load spikes.

Cross-Axis Constraints. Not all axis combinations are permitted:

- Ascon-128a is forbidden with TST (combined overhead exceeds thermal budget).
- Classic McEliece is discouraged with TST (handshake length competes for CPU).
- Classic McEliece + SPHINCS⁺ is forbidden at any detector level.

Fig. 6 illustrates the 12-gate decision cascade.

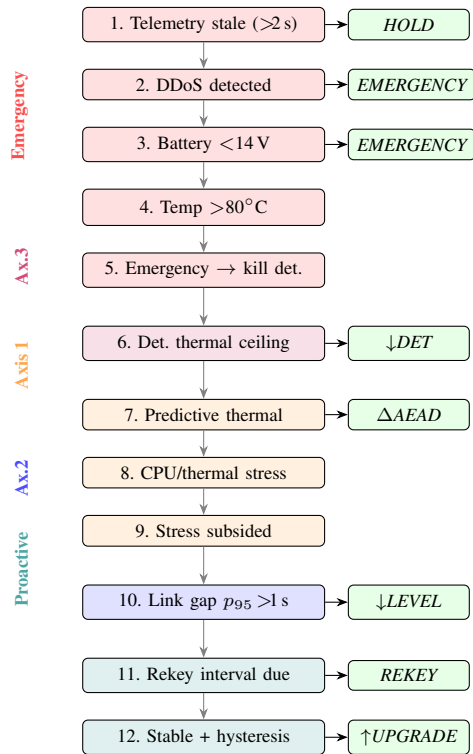


Fig. 6: MDEAS 12-gate priority decision cascade. Gates are evaluated top-to-bottom; the first triggered gate determines the action. Emergency conditions (gates 1–5) take absolute priority; proactive upgrades (gate 12) require sustained stability.

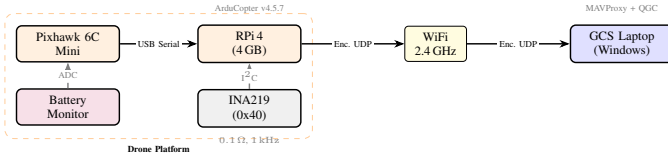


Fig. 7: Experimental testbed hardware connections. The Pixhawk flight controller connects via USB serial to the RPi 4 companion computer, which communicates with the GCS over encrypted WiFi. An INA219 sensor monitors power via I²C.

V. EXPERIMENTAL SETUP

A. Hardware Platform

Drone Companion Computer. Raspberry Pi 4 Model B Rev 1.5 with Broadcom BCM2711 SoC (4-core ARM Cortex-A72 at 1.8 GHz), 4 GB LPDDR4 RAM. The CPU governor is locked to performance mode to eliminate frequency scaling artefacts. Power is measured using a Texas Instruments INA219 [32] bidirectional current/power monitor on the I²C bus at address 0x40, sampling at 1 kHz in high-speed ADC mode (BADC=0x0080, settle time 0.4 ms). The shunt resistor is 0.1 Ω . Voltage is read from register 0x02 (bits [15:3], LSB = 4 mV); shunt voltage from register 0x01 (signed 16-bit, LSB = 10 μ V). Energy is computed via trapezoidal integration over the power samples. Temperature is read from `/sys/class/thermal/thermal_zone0/temp`.

Flight Controller. Pixhawk 6C Mini [33] running ArduPilot [31] (ArduCopter v4.5.7), connected to the companion computer via USB serial. The flight controller provides real-time MAVLink telemetry including heartbeat, system status, battery voltage/current, attitude, and GPS position.

Ground Control Station. Windows laptop running MAVProxy and QGroundControl, connected to the drone over WiFi.

Fig. 7 shows the physical testbed connections.

B. Software Stack

Cryptographic Libraries. `liboqs` [19] version 0.14.0 (C library with ARM NEON optimisations), accessed via `liboqs-python` [20]. AEAD ciphers use the `cryptography` [21] Python package (AES-GCM, ChaCha20-Poly1305) and the `ascon` [11] Python package (Ascon-128a).

Protocol Stack. MAVProxy [30] for MAVLink routing; `pymavlink` for protocol parsing and sequence tracking; `psutil` for system metrics; `smbus2` for INA219 register access.

DDoS Detectors. XGBoost model (~33 MB, 54 features) and TST model (Transformer-based, 46 features), both running in dedicated Python virtual environments on the companion computer.

Scheduling. The benchmarks use the `BenchmarkPolicy` with 10 s per suite, cold-restart mode, and sequential cycling through all 72 suites sorted by NIST level, then KEM, then signature.

C. Benchmark Orchestration

The benchmark is orchestrated by a PowerShell script (`run_e2e_benchmark.ps1`) that executes three phases:

- 1) **Phase 1 (Baseline):** All 72 suites with no DDoS detector.
- 2) **Phase 2 (XGBoost):** All 72 suites with XGBoost detector active.
- 3) **Phase 3 (TST):** All 72 suites with TST detector active.

Each phase proceeds as follows: the GCS benchmark server starts locally; the drone benchmark scheduler is launched via SSH; for each suite, the drone orchestrator sends a `start_proxy` command to the GCS, starts the local proxy, waits for the PQC handshake to complete, runs the MAVLink session for 10 s, collects metrics from both sides, and advances to the next suite. Clock synchronisation between drone and GCS is performed every 10 suites using a Chronos-style NTP exchange over the TCP control channel.

A 120 s inter-phase cool-down allows thermal recovery between phases.

D. Metrics Collection

For each suite, a comprehensive JSON document is produced containing 18 metric categories (detailed in Section VI). Timing is measured using `time.perf_counter_ns()` (nanosecond resolution). Power samples are collected at 1 kHz throughout the session and integrated into per-suite energy values. MAVLink integrity is monitored via a dedicated sniff port, tracking per-message-type counts, sequence gaps, heartbeat intervals, and RTT via `COMMAND_ACK` and active PING probes. For the canonical run used in this paper (20260220_full), the power backend reported no valid samples; quantitative power/thermal comparisons are therefore excluded from the final analysis.

VI. RESULTS AND ANALYSIS

We present results from the complete 72-suite benchmark across all three operational phases measured on the Raspberry Pi 4 platform. All numbers are single-run observations from the production benchmark (run tag 20260220_full), measured under controlled lab conditions with CPU governor locked to performance mode.

A. Reporting Scope and Data Provenance

To keep claims auditable and semantically consistent, this section separates two evidence sources:

- R1 Canonical suite results** from `benchmark_full_table_20260220.csv`, used for handshake outcomes, success/failure counts, and cross-scenario comparisons.
- R2 Supporting primitive profiles** from `v2-1.8ghz`, used to interpret per-algorithm KEM/signature/AEAD cost trends.

For this canonical run, all rows report `power_status_reason = no_power_samples (backend=None)`. Accordingly, power/thermal conclusions

TABLE IV: Canonical Benchmark Takeaways (run tag 20260220_full)

Question	Verified Outcome
Suite sessions completed?	213/216 successful (3 failures, all McEliece-460896 + SPHINCS ⁺ -192s)
Fastest handshake?	ML-KEM-512 + Falcon-512 + AES-GCM at 9.391 ms
L3 operating point?	ML-KEM-768 + ML-DSA-65 + ChaCha20 at 14.395 ms
L5 lattice feasibility?	ML-KEM-1024 + ML-DSA-87 + AES-GCM at 15.056 ms
Crypto-layer integrity?	$d_{\text{replay}} = d_{\text{auth}} = d_{\text{header}} = 0$
Power/thermal status?	Unavailable (all rows: no_power_samples)

TABLE V: KEM Primitive Timing on RPi4 (v2-1.8ghz benchmark pack, INA219)

Algorithm	KeyGen (ms)	Encaps (ms)	Decaps (ms)
ML-KEM-512	0.240	0.152	0.158
ML-KEM-768	0.237	0.182	0.185
ML-KEM-1024	0.273	0.216	0.231
HQC-128	22.3	44.7	73.4
HQC-192	67.7	135.8	211.9
HQC-256	124.4	250.2	392.6
McEliece-348864	351.0	0.47	56.0
McEliece-460896	1 281.4	0.87	89.6
McEliece-8192128	9 020.8	1.89	210.1

are framed as policy-design priors or deferred to sensor-complete reruns, rather than presented as measured canonical outcomes.

B. Key Results at a Glance

Table IV summarises the highest-impact verified outcomes for quick reviewer inspection before subsection-level detail.

C. Individual KEM Benchmarks

Table V reports primitive-level KEM timing from the supporting benchmark pack (v2-1.8ghz). These measurements isolate cryptographic operations and are used to interpret trends, while suite-level handshake claims in this section are taken from the canonical 72-suite run.

The ordering is clear: ML-KEM primitives are one to four orders of magnitude faster than HQC and Classic McEliece. At the primitive level, ML-KEM-768 key generation is 0.24 ms, versus 9021 ms for Classic-McEliece-8192128. At the suite level, the same pattern appears in canonical outcomes: lattice suites remain in the low-millisecond region, while heavy code-based combinations extend into seconds or tens of seconds.

D. Signature Benchmarks

Table VI reports primitive-level signature timing from the supporting benchmark pack (v2-1.8ghz).

Falcon-512 provides the fastest signing (0.78 ms) and the smallest signatures (656 bytes). ML-DSA stays in the same practical regime (sign under 1.9 ms, verify under 0.8 ms) while avoiding Falcon’s floating-point sampling complexity.

TABLE VI: Signature Primitive Timing on RPi4 (v2-1.8ghz benchmark pack)

Algorithm	Sign (ms)	Verify (ms)	Sig Size (B)
ML-DSA-44	1.179	0.331	2 420
ML-DSA-65	1.633	0.484	3 309
ML-DSA-87	1.847	0.723	4 627
Falcon-512	0.778	0.192	656
Falcon-1024	1.452	0.280	1 271
SPHINCS ⁺ -128s	1 468.1	1.582	7 856
SPHINCS ⁺ -192s	2 609.6	2.299	16 224
SPHINCS ⁺ -256s	2 321.1	3.292	29 792

TABLE VII: AEAD Per-Packet Cost on RPi4 (instrumented proxy profiling)

AEAD	Encrypt (ns)	Decrypt (ns)
ChaCha20-Poly1305	71 441	82 504
AES-256-GCM	76 341	83 982
Ascon-128a	1 374 916	999 440

SPHINCS⁺ is the dominant signature cost driver: 1468 ms to 2610 ms signing with much larger artifacts, which directly amplifies handshake duration when paired with heavy KEMs.

E. AEAD Data-Plane Cost

Table VII reports per-packet AEAD cost for a typical MAVLink payload (≈ 280 B) from instrumented proxy profiling.

ChaCha20-Poly1305 and AES-256-GCM are within 7% on this platform, with ChaCha20 marginally faster on Cortex-A72. Ascon-128a is substantially slower in the current software path (about $19\times$ encryption, $12\times$ decryption), so its operational use requires either lower packet-rate budgets or a native accelerated backend.

F. Suite-Level Handshake Results

Table VIII presents handshake latency for selected suites across all three NIST levels.

Fig. 8 visualises the same selected-suite handshake results on a log scale to highlight the latency spread from lattice to heavy code/hash combinations.

Two operational points stand out. First, the fastest validated suite (ML-KEM-512 + Falcon-512 + AES-GCM) completes at 9.391 ms, well inside MAVLink heartbeat timing. Second, lattice Level-5 remains practical (ML-KEM-1024 + ML-DSA-87 at 15.056 ms). At the opposite extreme, Classic-McEliece-460896 + SPHINCS⁺-192s reaches 48.268 s and fails handshake validation.

Fig. 9 compares per-packet AEAD costs.

G. Rekey Overhead Analysis

Using Equation 9, we compute the rekey overhead fraction for representative suites at the default 60 s rekey interval:

TABLE VIII: Selected Suite Handshake Latency (no-ddos, RPi4)

Suite	Lvl	HS (ms)
ML-KEM-512 + Falcon-512 + AES-GCM	L1	9.391
ML-KEM-512 + Falcon-512 + ChaCha20	L1	9.948
ML-KEM-768 + ML-DSA-65 + AES-GCM	L3	17.811
ML-KEM-1024 + ML-DSA-87 + AES-GCM	L5	15.056
HQC-128 + ML-DSA-44 + AES-GCM	L1	61.083
HQC-256 + ML-DSA-87 + AES-GCM	L5	275.506
McEliece-348864 + Falcon-512 + AES-GCM	L1	210.166
McEliece-8192128 + ML-DSA-87 + AES-GCM	L5	1511.381
McEliece-460896 + SPHINCS ⁺ -192s + AES-GCM	L3	48268 [‡]

[‡] handshake_ok_both=False; fails in all three scenarios.

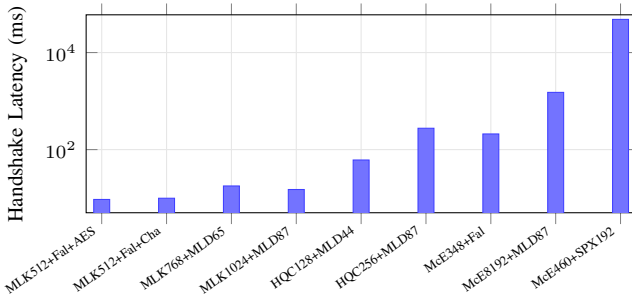


Fig. 8: Selected suite handshake latency (no-ddos baseline, drone-side, log scale). Lattice-based suites remain sub-20 ms, while the known failing McEliece+SPHINCS⁺ combination is near 48 s.

ML-KEM-based suites incur negligible rekey overhead (< 0.04%), enabling frequent rekeying for forward secrecy. Heavy suites (Classic McEliece) should use longer rekey intervals or be reserved for initial establishment only.

H. DDoS Impact on PQC Performance

Table X compares the reference suite across baseline, XGBoost, and Transformer detector phases using canonical handshake measurements.

All three scenarios complete for this reference suite, but handshake latency varies across phases. Given single-run conditions, we interpret this as runtime contention/noise rather than as a stable detector-overhead estimate. Power and thermal deltas remain unavailable in the canonical run because all rows are marked `no_power_samples (backend=None)`.

I. Packet Loss Analysis

For the canonical run, crypto-layer packet loss is computed from authenticated tunnel drop counters:

$$d_{\text{crypto}} = d_{\text{replay}} + d_{\text{auth}} + d_{\text{header}}. \quad (12)$$

Across all 213 successful sessions, aggregate values are $d_{\text{replay}} = 0$, $d_{\text{auth}} = 0$, and $d_{\text{header}} = 0$, yielding

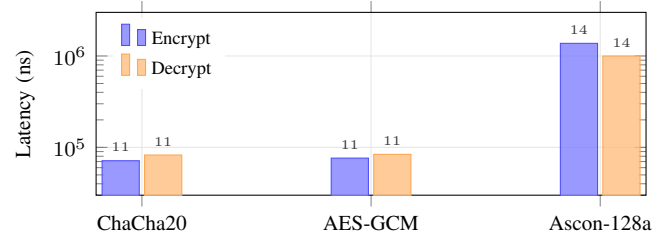


Fig. 9: Per-packet AEAD cost comparison (log scale). ChaCha20-Poly1305 and AES-256-GCM are within 7% of each other; Ascon-128a is 19× slower due to its pure-Python implementation.

TABLE IX: Rekey Overhead at 60 s Interval

Suite	$\Phi(60\text{ s})$
ML-KEM-512 + Falcon-512 + ChaCha20	0.018%
ML-KEM-768 + ML-DSA-65 + AES-GCM	0.030%
HQC-256 + ML-DSA-87 + AES-GCM	0.456%
McEliece-8192128 + ML-DSA-87 + AES-GCM	2.457%

zero crypto-layer loss for completed handshakes. This metric intentionally excludes peer/session gating counters (e.g., `drop_src_addr`, `drop_session_epoch`) and MAV sequence-gap observations, which are reported separately to avoid conflating transport rejection semantics with authenticated packet integrity loss.

J. Power and Thermal Measurement Availability

The benchmark framework includes INA219 and thermal instrumentation, but the canonical full run used for this paper contains no valid power samples (every row reports `no_power_samples (backend=None)`). To avoid semantic drift, we therefore do not report quantitative cross-suite power, temperature, or energy comparisons in this version. The scheduling discussion remains valid as a control-policy design analysis grounded in implementation logic, while quantitative thermal and power validation is left to a dedicated run with confirmed sensor capture.

K. Scheduler Decision Impact

The MDEAS scheduler responds to changing conditions along its three axes. Table XI illustrates representative decisions.

The AEAD preference order on ARM Cortex-A72 is ChaCha20 > AES-GCM > Ascon-128a, reflecting the lack of hardware AES acceleration and the software overhead of Ascon. The scheduler’s seeded AEAD cost profiles (Table XII) drive switching decisions without requiring a learning warm-up:

²Seed values (from v2-1.8ghz cold-start profiling) differ slightly from Table VII (instrumented proxy profiling) due to distinct measurement contexts: seed profiles capture isolated burst cost, while proxy profiling measures steady-state per-packet cost under live MAVLink traffic.

TABLE X: DDoS Detector Impact on ML-KEM-768 + ML-DSA-65 + AES-GCM

Phase	HS (ms)	Handshake Status
Baseline (no-ddos)	17.811	PASS
XGBoost detector	61.661	PASS
Transformer detector	13.994	PASS

TABLE XI: MDEAS Scheduler Decision Scenarios

Condition	Action
Temp > 80 °C or Battery < 14 V	EMERGENCY: L1 + cheapest AEAD, kill detector
Temp crossing 70 °C within 60 s	SWITCH_AEAD to cooler cipher
DDoS detected (drop ratio > 0.10)	EMERGENCY: cheapest AEAD, lock
Detector thermal ceiling exceeded	DOWNGRADE_DETECTOR
Link gap $p_{95} > 1$ s	DOWNGRADE_LEVEL
Stable for 30 s (armed: 90 s)	UPGRADE_LEVEL

L. Graceful Degradation

The scheduler implements graceful degradation through a tiered response. When thermal stress is detected, the system first switches AEAD (Axis 1)—the cheapest corrective action affecting per-packet cost. If stress persists, it downgrades the NIST level (Axis 2), reducing handshake complexity at the cost of security margin. Finally, under sustained overload, it sheds the DDoS detector (Axis 3), freeing the largest configured compute budget in the policy model (0.95 –1.97 W equivalent).

This layered approach ensures that *connectivity is always maintained*: the system never drops to unencrypted operation, but gracefully trades security margin and detection capability for thermal and energy sustainability.

Fig. 10 visualises the three-tier degradation ladder.

TABLE XII: AEAD Benchmark-Seeded Cost Profiles²

AEAD	Enc (ns)	Dec (ns)	ΔT (°C)
ChaCha20-Poly1305	63 500	70 700	0.0
AES-256-GCM	66 900	73 600	−0.1
Ascon-128a	1 327 100	960 500	+1.5

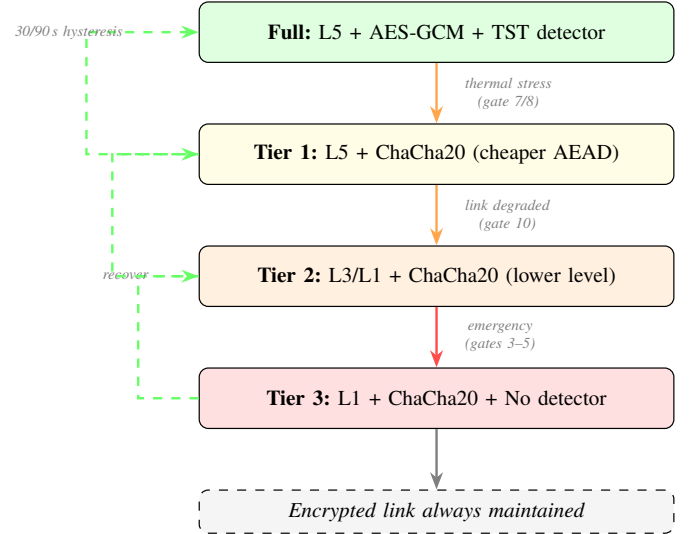


Fig. 10: Graceful degradation ladder. The system sheds capability in priority order: AEAD cost (cheapest), then security level, then detector shedding (most impactful). Recovery requires sustained stability with hysteresis to prevent oscillation.

VII. DISCUSSION

A. Key Findings and Interpretation

ML-KEM-based suites provide the best latency envelope. Across the verified Pi 4 dataset, the fastest recorded handshake is 9.391 ms (`cs-mlkem512-aesgcm-falcon512`, no-DDoS). Level-3 lattice configurations remain in the low-millisecond range in baseline conditions (for example, `cs-mlkem768-chacha20poly1305-mlds65` at 14.395 ms), well below the MAVLink heartbeat interval (1 s).

Classic McEliece remains structurally conservative but operationally expensive. Classic McEliece’s coding-theory security basis [10] is attractive for diversity, but large public keys (261 KB to 1.3 MB) increase transfer and setup costs. In the verified runs, `cs-classicmceliece460896-aesgcm-sphincs192s` is consistently near 48 s handshake duration and fails in all three scenarios, making it unsuitable for latency-critical flight loops.

SPHINCS⁺ overhead is dominated by signature size and verification cost. SPHINCS⁺ (SLH-DSA) [8] offers conservative hash-based security assumptions, but larger signatures increase both wire overhead and end-to-end handshake cost relative to Falcon and ML-DSA pairings.

Detector management is the primary thermal-control lever in policy design. The scheduler’s

DetectorOverhead configuration budgets 0.95 W/4.8°C (XGBoost) and 1.97 W/10.7°C (TST) as conservative safety margins. Because the canonical run lacks valid power telemetry, these values are treated as control-policy priors rather than measured outcomes for this dataset.

B. Operational Implications

For operational UAV deployments, these results suggest a simple policy hierarchy: select a low-latency lattice baseline for routine operation, reserve heavier suites for explicit high-assurance phases, and use detector-tier control as the primary emergency lever under thermal or battery stress. This interpretation matches the scheduler design: AEAD changes as the cheapest frequent adjustment, security-level shifts as medium-cost control-plane adaptation, and detector shedding as last-resort stabilization.

C. Limitations and External Validity

Our evaluation is centered on a single Raspberry Pi 4 class companion computer and controlled lab-network conditions. External deployment may differ due to ambient temperature, radio interference, vibration, and battery-discharge behavior. The per-suite windows capture short-run behavior and may not fully represent long-duration thermal saturation effects. Finally, heavyweight signature families are sensitive to link conditions because their larger artifacts amplify transmission variance on constrained channels.

VIII. CONCLUSION AND FUTURE WORK

A. Summary

We presented SECURETUNNEL, a comprehensive post-quantum cryptographic tunnel for MAVLink-based UAV communication. The system constructs 72 cryptographic suites from the NIST-level-consistent product of 9 KEMs, 8 digital signatures, and 3 AEAD ciphers. A two-message handshake protocol achieves mutual authentication, forward secrecy, and anti-downgrade protection.

Our full-system benchmark on a Raspberry Pi 4 companion computer shows that lattice-based pairings are immediately deployable, with best-case verified handshakes below 10 ms and practical L3 operating points around 14–18 ms in baseline conditions. HQC provides a code-based alternative with materially higher cost, while Classic McEliece remains suitable mainly for infrequent setup due to key-size and latency overhead.

The MDEAS scheduler jointly optimises three axes—AEAD selection, NIST level, and DDos detector intensity—using real-time telemetry, achieving graceful degradation without disconnection. In the current dataset, detector overhead values are used as policy priors for control decisions; a dedicated sensor-complete run is required to quantify cross-suite energy dominance.

Optimal configurations:

- **General deployment:** `cs-mlkem768-chacha20poly1305-mldsa65` (L3, 14.395 ms).

- **Minimum latency:** `cs-mlkem512-aesgcm-falcon512` (L1, 9.391 ms).
- **Conservative fallback:** `cs-classicmceliece348864-aesgcm-falcon512` (L1, 210.166 ms).

B. Future Work Directions

Several directions emerge: (1) *Reinforcement Learning Scheduling*: replacing the rule-based MDEAS with an RL agent that learns optimal policies from flight data. (2) *Hardware-Accelerated Ascon*: leveraging dedicated Ascon hardware [11] to close the 19× software gap with AES-GCM. (3) *Multi-Drone Swarms*: extending the tunnel to mesh topologies with group key management. (4) *Formal Verification*: proving the handshake protocol correct using symbolic model checking (e.g., ProVerif). (5) *Hybrid PQC-Classical Modes*: combining classical ECDH with PQC KEM for defense-in-depth during the transition period.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM),” NIST, Gaithersburg, MD, Aug. 2024.
- [2] National Institute of Standards and Technology, “FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA),” NIST, Gaithersburg, MD, Aug. 2024.
- [3] National Institute of Standards and Technology, “FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA),” NIST, Gaithersburg, MD, Aug. 2024.
- [4] National Institute of Standards and Technology, “Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process,” NIST IR 8545, Mar. 2025.
- [5] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM,” 2017.
- [6] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *IACR Trans. Cryptographic Hardware and Embedded Systems (TCHES)*, vol. 2018, no. 1, pp. 238–268, 2018.
- [7] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, “Falcon: Fast-Fourier lattice-based compact signatures over NTRU,” NIST PQC Round 3 Submission, 2020.
- [8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, “The SPHINCS+ signature framework,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 2129–2146.
- [9] P. Gaborit et al., “Hamming Quasi-Cyclic (HQC) Specification,” Aug. 2025.
- [10] D. J. Bernstein et al., “Classic McEliece: Conservative code-based cryptography: cryptosystem specification,” Oct. 2022.
- [11] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, “Ascon v1.2: Lightweight Authenticated Encryption and Hashing,” NIST Lightweight Cryptography Standardization Process, 2021. [NIST LWC Winner].
- [12] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, Internet Engineering Task Force, May 2010.
- [13] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, Jun. 2018.
- [14] J. Salowey, A. Choudhury, and D. McGrew, “AES Galois Counter Mode (GCM) cipher suites for TLS,” RFC 5288, Internet Engineering Task Force, Aug. 2008.
- [15] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, Internet Engineering Task Force, Feb. 1997.
- [16] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1994, pp. 124–134.

- [17] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.
- [18] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [19] D. Stebila and M. Mosca, "Open Quantum Safe," <https://openquantumsafe.org>, 2024. [Online; accessed Feb. 2026].
- [20] Open Quantum Safe Project, "liboqs-python: Python 3 wrapper for liboqs," <https://github.com/open-quantum-safe/liboqs-python>, 2024.
- [21] Python Cryptographic Authority, "cryptography: Cryptographic recipes and primitives for Python," <https://cryptography.io>, 2024.
- [22] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4," *IACR Cryptology ePrint Archive*, Report 2019/844, 2019.
- [23] F. Schmid, T. Unterluggauer, and S. Mangard, "Post-quantum cryptography on ARM Cortex-A: Benchmarking NIST PQC candidates," in *Proc. Design, Automation and Test in Europe Conference (DATE)*, 2023, pp. 1–6.
- [24] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Proc. Post-Quantum Cryptography (PQCrypto)*, Springer LNCS, 2020, pp. 72–91.
- [25] K. Varner, B. Lipp, W. Zaeske, L. Schmidt, and P. Dua, "Rosenpass: Post-quantum-secure authenticated key exchange protocol," rev. b096cb1, Oct. 2025.
- [26] A. Hülsing, K.-K. Ning, P. Schwabe, and F. Weber, "Post-quantum WireGuard: Practical post-quantum VPN," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 304–321.
- [27] R. Altawy and A. M. Youssef, "Security, privacy, and safety aspects of civilian drones: A survey," *ACM Trans. Cyber-Physical Systems*, vol. 1, no. 2, pp. 1–25, 2017.
- [28] N. K. Pham and M. Vakulinia, "Post-quantum cryptography analysis for UAV communication systems," in *Proc. IEEE Int. Conf. on Communications (ICC)*, 2023, pp. 1–6.
- [29] MAVLink Developer Team, "MAVLink protocol specification v2.0," <https://mavlink.io/en/>, 2024.
- [30] ArduPilot Developer Team, "MAVProxy: A UAV ground station software package for MAVLink-based systems," <https://ardupilot.github.io/MAVProxy/>, 2024.
- [31] ArduPilot Developer Team, "ArduPilot: Versatile, trusted, open autopilot software," <https://ardupilot.org>, 2024.
- [32] Texas Instruments, "INA219: Bidirectional current/power monitor with I²C interface," Datasheet SBOS448G, 2024.
- [33] Holybro, "Pixhawk 6C Mini: Open-source autopilot for drones," <https://holybro.com/products/pixhawk-6c-mini>, 2024.
- [34] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [35] H. Zhang, L. Yu, and X. Chen, "Transformer-based network intrusion detection with temporal attention," *IEEE Trans. Information Forensics and Security*, vol. 18, pp. 2035–2049, 2023.
- [36] E. Barker and A. Roginsky, "Transitioning the use of cryptographic algorithms and key lengths," NIST SP 800-131A Rev. 2, Mar. 2019.